Sarah OUNES **ESILV A5**
Louis-Marie MATTHEWS **CDOF 3**

Graph and Mining

Project Report

2025-2026

Contents

1 Introduction **2**

2 Exploring and preparing the dataset **3**
2.1 Basic EDA 3
2.2 Preparing the data 4

3 Importing the data into Neo4j Desktop **4**

4 Exploring the data with Neo4j **4**
4.1 Creating a projection 4
4.1.1 Creating a bipartite graph 7
4.1.2 Creating a projection for a monopartite graph 8
4.2 Accessing the projections we created 9

5 Analysis **9**
5.1 Similarity Analysis 9
5.1.1 Jaccard Similarity 9
5.1.2 Overlap Similarity 12
5.2 Community Analysis 13
5.2.1 Louvain 13
5.2.2 Connected Components 14
5.3 Degree Centrality 14
5.4 Link Prediction 16
5.5 Path finding 18

6 Conclusion **20**

1 Introduction

The objective of this project is to explore and analyse a real-world women's basketball dataset using graph-based modeling and analysis techniques. By representing players, teams, awards, and educational backgrounds as nodes and relationships, we aim to better understand career paths, similarities between players, and structural patterns within the data through graph algorithms.

All the code, data preprocessing scripts, and supporting materials used in this project are available in the public GitHub repository: `https://github.com/matounes/basket`. In particular, the exploratory data analysis (EDA) of the original dataset was carried out in a dedicated Jupyter notebook, which is included in the repository. This notebook documents the initial inspection of the data, its structure, and the steps that motivated the subsequent graph modeling choices.

2 Exploring and preparing the dataset

2.1 Basic EDA

The dataset we are using is the `Basketball_women.json` dataset. As for any data science project, we begin with an EDA[1]

For this, we use Python with the Pandas library, as well as a notebook as it makes prototyping and seeing the result very fast and convenient. We use these tools to get an idea of the structure and the volume of the data.

```
1  import pandas as pd
2  import numpy as np
```

```
1  df = pd.read_json('data/original/Basketball_women.json', lines=True)
2
3  df.iloc[0]
```

```
_id                                                    abrahta01w
firstName                                                 Tajama
middleName                                                Ethia
lastName                                                Abraham
fullGivenName                                     Tajama Abraham
nameNick                                                  TJ,Taj
pos                                                            C
height                                                        74
weight                                                       190
college                                          George Washington
birthDate                                               1975-09-27
birthCity                                                St,Croix
birthCountry                                                  ISV
highSchool                                              Kecoughtan
hsCity                                                     Hampton
hsState                                                         VA
hsCountry                                                      USA
players_teams      [{'year': 1997, 'tmID': 'SAC', 'games': 28, 'm...
awards_players                                                 []
Name: 0, dtype: object
```

Figure 1: We use Pandas for very basic EDA.

By looking at the first row, as well as at the result of `df.info()` and the non-empty values for the columns `players_teams` and `awards_players`, we can identify several distinct entities that will our constitute the types of the future nodes of our graph. Those are:

- Player,
- Award,

[1] Exploratory Data Analysis.

- College,
- High-School,
- and Team.

Player is the only node with a one-way relationship towards all other node types. In certain cases, as for its relationship with `Team`, the relationship has parameters (e.g. the year, the number of goals, etc).

2.2 Preparing the data

We split the dataset into 5 smaller CSV files. CSV files are easier to import into Neo4j Desktop. Each dataset corresponds to a node type. The Player node type is the only one with properties, all the others only contain an identifier for the specific node. This means we do not need to create datasets for relationships with the Player node type, we can simply package them with the node type.

Those datasets are:

- `awards.csv`: A list of awards, and the relationships between players and awards, as well as relationship information (when was the award won by the player, etc).
- `colleges.csv`
- `high_schools.csv`
- `players_teams.csv`: Similar to `awards.csv`, but for teams.
- `players.csv`: Lists players information, and their high school and college they attended.

3 Importing the data into Neo4j Desktop

Neo4j Desktop makes it convenient and easy to import data. From its graphical user interface, and after creating an instance and a database for our project, it sufficed to import the CSV files, and determine the different node labels (the name of the type), properties, as well as the relationships.

4 Exploring the data with Neo4j

With 1878 nodes and relationships, the dataset is huge, and not everything can be seen from the Explore section of Neo4j Desktop. (Fig. 4)

4.1 Creating a projection

First, we must understand what a **projection** is. This is a very important concept. It can be useful to compare them to **Views** in traditionnal RDBMS. They are *in-memory* graphs, made from a subset of nodes and relationships of the database graph, and sometimes transformed. As they exist in the DBMS' memory only, they are erased when the instance shuts down. This means they are not part of the database's files.

Projections are not meant to be used like views however. They are not intented to be displayed in Neo4j Desktop's Explorer for instance. They are intended to facilitate the running of algorithms on our data. Indeed, because they are in-memory and not on disk, algorithms

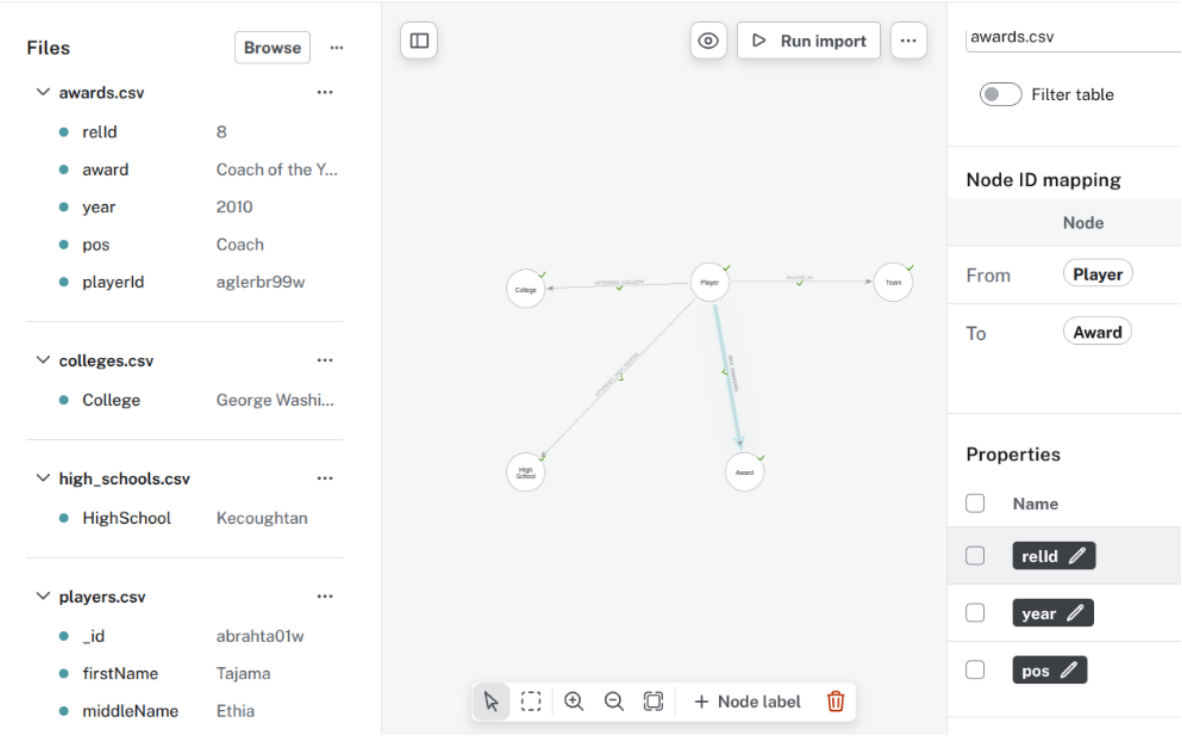

Figure 2: Importing data into Neo4j is intuitive.

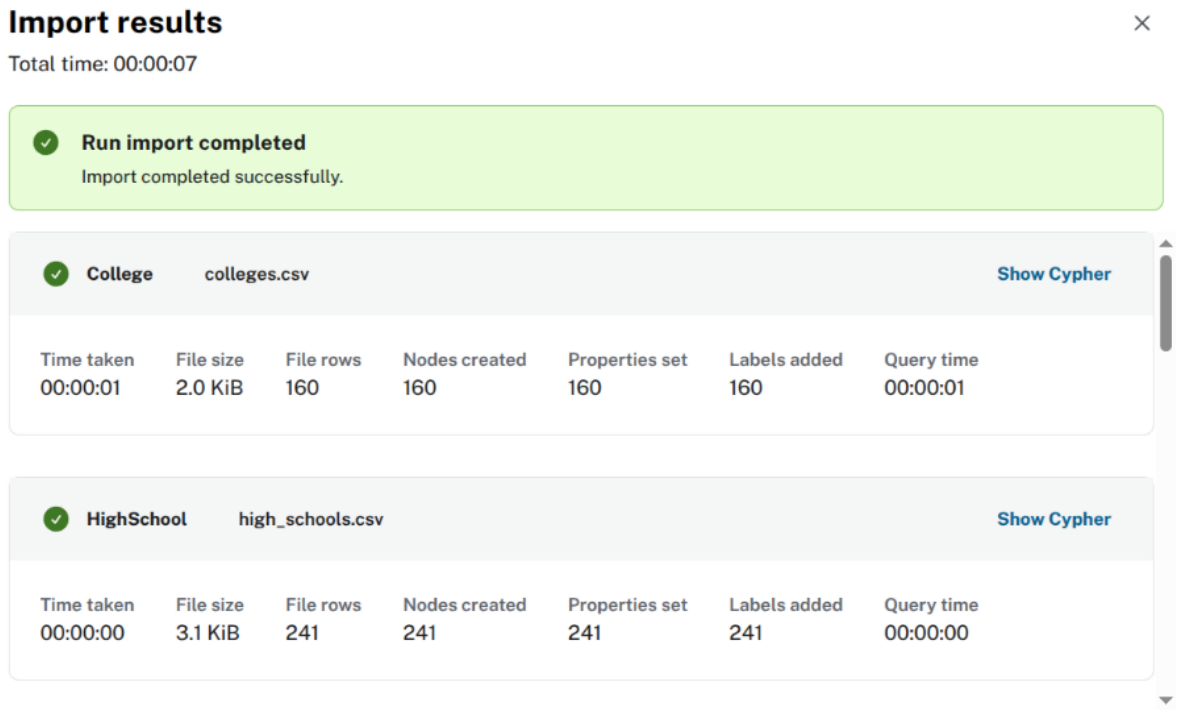

Figure 3: Importing data is successful.

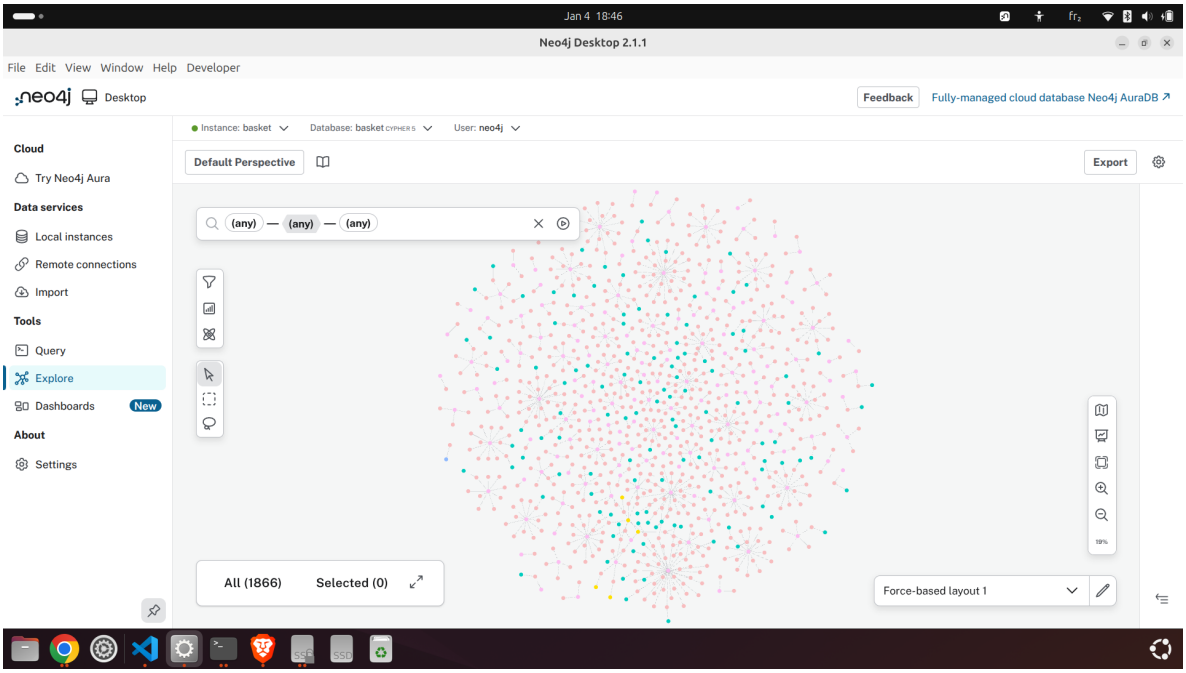

Figure 4: The dataset is huge and not everything can be shown at the same time.

can run faster. In addition, projections are decoupled from the database, which prevents accidental modification and makes it possible to create projections with certain caracterisics, i.e. only considering players from a certain school for instance.

Now that we understand what a projection is and what purpose they serve, let's create one in Neo4j.

```
1 MATCH (u:Player)-[r:PLAYED_IN]->(t:Team)
2 WHERE r.year > 2010
3 WITH gds.graph.project("recent_players", u, t) AS g
4 RETURN g.graphName AS graph, g.nodeCount AS nodes, g.relationshipCount AS rels
```

Table RAW

graph	nodes	rels
"recent_players"	191	217

Figure 5: Creating a projection named recent_players

Here, we create a projection based on the result of a **Cypher**[2] **query**. This is done by passing the output of the query, which always consists of a table of rows even when shown as a graph, to a **Procedure** from the **GDS plugin**. The GDS plugin then creates a projection (hence, "project") from the query's result.

We introduced three important terms: **Procedure**, **Plugin**, and **GDS**.

- A **Procedure** is a function that can modify the database or run advanced algorithms. They are not part of the Cyper DSL. Instead, they are provided by plugins and can be

[2]Neo4j's DSL

called from within a Cypher query. (More on that later.)

- A **Plugin** extends the functionalities of Neo4j. Its installation is tied to a specific instance, i.e. a Neo4j server, and not to a database or to a Neo4j Desktop installation.
- **GDS** is an official plugin that provides advanced algorithms for analysing graphs.

In the following code snippet, we create a projection of the players with a team and who were awarded at least one award:

```
MATCH (u:Player)-[r:PLAYED_IN]->(t:Team)
WHERE (u)-[:WAS_AWARDED]->(:Award)
WITH gds.graph.project("recent_players", u, t) AS g
RETURN g.graphName AS graph, g.nodeCount AS nodes, g.relationshipCount AS
rels
```

There are simpler and more useful manners to create projections, for instance, using GDS' native projection system:

```
CALL gds.graph.project("bipartite", ["Player", "Team"], {PLAYED_IN: {
    orientation: 'UNDIRECTED'}})
```

We have `undirectedRelationshipTypes` to make relationships undirected. Certain algorithms and metrics, such as the calculation of node degrees, only consider certain edge orientations. In addition, because there is only one edge type which only indicates a relationship between two different node types, there is no orientation.

This creates a projection with all players and all their relationships with teams. This effectively turns our k-partite graph into a bipartite one. But, gee, what is a bipartite graph?

4.1.1 Creating a bipartite graph

The graph we created by importing the CSVs that were transformed using Pandas is a **k-partite graph**. This is because we have several types of nodes and edges only between types. Certain algorithms only run on monopartite or bipartite graphs however, meaning graphs with one node type and relationship type in the former case and two partitions and with edges only between those partitions in the latter case. [3]

In general, as we lower the k of our k-partite graph, we lose information gradually. For this reason, we will first make a bipartite projection from our graph, then a monopartite one.

We create the bipartite graph by only condidering the "Player" and the "Team" nodes, as they are the relationships with the most information. High school and college are already stored as properties of the "Player" node. [4]

So, creating our bipartite graph is just a matter of only creating the "Player" and "Team" nodes. We do this by calling the `gds.graph.project()` **procedure** of the GDS **plugin**, which results in a new **projection**[5]. (Fig. 6)

[3] "Most graph algorithms are designed to be executed on a monopartite network, where only a single node and relationship type are present." `https://www.oreilly.com/library/view/graph-algorithms-for/9781617299469/OEBPS/Text/ch06.htm`

[4] We could decide to create relationships between players who share the same high school and college, but not only do we not lose information yet (these informations are still stored in the node's properties), this could also result in the creation of artificial edges and huge, misleading cliques.

[5] Oh yeah, the GDS Plugin's Procedure to create a Projection! You Sir, are a mouthful!

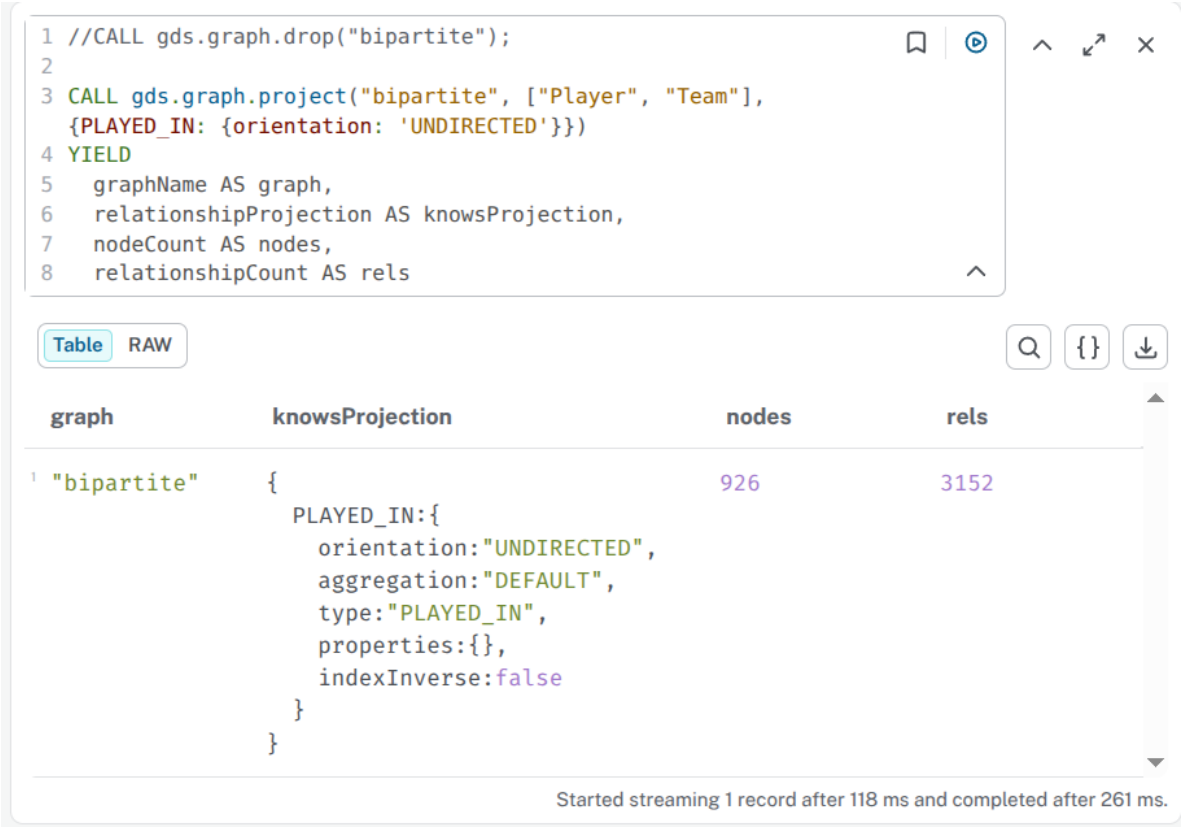

Figure 6: Creating a bipartite graph

4.1.2 Creating a projection for a monopartite graph

We will now create our monopartite graph as a projection of the bipartite graph.

To do so, we will only retain Player nodes, and create relationships between them, all that using a projection. [6]

```
MATCH (source:Player)
OPTIONAL MATCH (source)-[:PLAYED_IN]->(t)<-[:PLAYED_IN]-(target)
WITH gds.graph.project(
    'monopartite',
    source,
    target,
    {},
    {undirectedRelationshipTypes: ['*']}
) as g
RETURN g.graphName AS graph, g.nodeCount as nodes, g.relationshipCount as rels
```

Notice that we have `MATCH` and `OPTIONAL MATCH`. This allows us to capture both **connected** and **unconnected** nodes.

Again, we have `undirectedRelationshipTypes` to make relationships undirected.

We get, as a result, 893 nodes (the number of players) with 211192 relations in total. The extreme increase of relationships (compared to the bipartite and original graph) is to be expected when projecting to a lower partiteness. This is an indication of artificial relationships which might result in large, hard-to-exploit cliques.

[6] Notice we use the new Cypher projection syntax, as the old one "graph.gds.project.cypher()" is deprecated.

4.2 Accessing the projections we created

We can then access the list of projections we created. (Figure 7.)

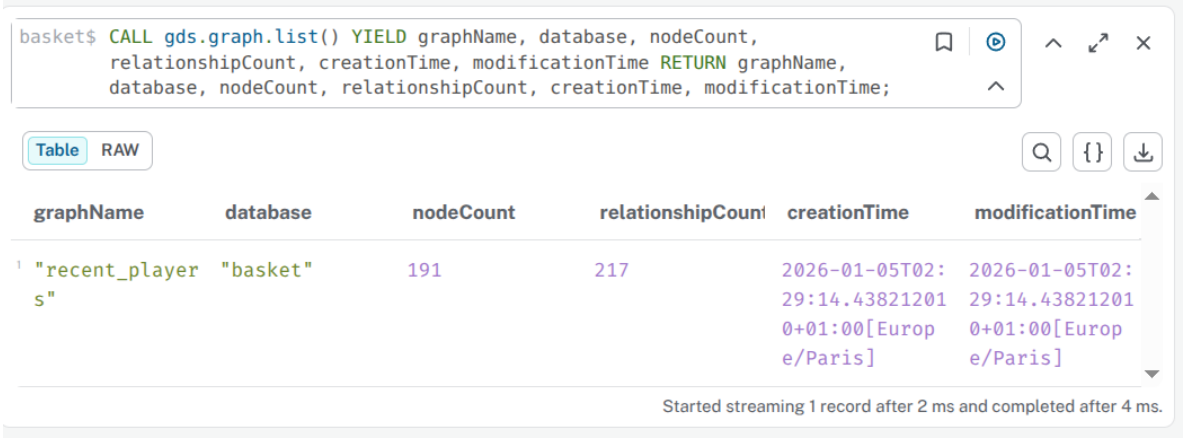

Figure 7: We can see the list of projections.

5 Analysis

5.1 Similarity Analysis

A similarity function can be defined as a function that takes two input x and y, and that ouptuts a score, generally between 0 and 1 (though sometimes between -1 and 1) determining how close the two inputs are.

Several similarity functions are implemented in GDS.

5.1.1 Jaccard Similarity

This similarity functions measures overlap between sets. When applied to a graph such as ours, the sets are the respective nodes' set of neighbours. Meaning that if for two nodes we get a score of 1.0, those two nodes have exactly the same neighbours.

We will compute the Jaccard Similarity for the top 10 nodes (in terms of degree[7]) of each of our projection (bipartite and monopartite).

For the bipartite graph, we get the following result. (Fig. **??**)

```
// Top-10 Players by degree (to Teams) + Jaccard similarity (Players vs
    Players)
CALL {
  CALL gds.degree.stream(
    'bipartite',
    { nodeLabels: ['Player'], relationshipTypes: ['PLAYED_IN'] }
  )
  YIELD nodeId, score
  RETURN collect(nodeId)[0..10] AS topPlayers
}
CALL gds.nodeSimilarity.filtered.stream(
  'bipartite',
  {
    sourceNodeFilter: topPlayers,
    targetNodeFilter: 'Player',
```

[7] Number of relationships.

```
    similarityMetric: 'JACCARD',
    topK: 20,
    similarityCutoff: 0.1
  }
)
YIELD node1, node2, similarity
RETURN
  gds.util.asNode(node1).fullGivenName AS player,
  gds.util.asNode(node2).fullGivenName AS similarPlayer,
  similarity
ORDER BY similarity DESC, player;
```

```
25 ORDER BY similarity DESC, player;
26
```

Table RAW

	player	similarPlayer	similarity
61	"Matee Ajavon"	"Nyree Roberts"	1.0
62	"Matee Ajavon"	"Tammy Jackson"	1.0
63	"Tajama Abraham"	"Stacy Clinesmith"	1.0
64	"Svetlana Abrosimova"	"Renee Montgomery"	0.6666666666666666
65	"Svetlana Abrosimova"	"Lindsay Whalen"	0.6666666666666666
66	"Svetlana Abrosimova"	"Shona Thorburn"	0.6666666666666666
67	"Svetlana Abrosimova"	"Janell Burse"	0.6666666666666666
68	"Tajama Abraham"	"Crystal Kelly"	0.6666666666666666
69	"Tajama Abraham"	"Kristin Haynie"	0.6666666666666666
70	"Danielle Adams"	"Gwen Jackson"	0.5

Started streaming 140 records after 154 ms and completed after 189 ms.

Figure 8: We compute the Jaccard on the bipartite projection.

The similarity score goes gradually from 1.0 to 0.33. This indicates that all of the top 10 nodes share at least one common neighbor. In addition, a lot of scores are 1.0, meaning two players share exactly the same team history (regardless of the dates).

Now, with the monopartite graph:

```
CALL {
  CALL gds.degree.stream(
    'monopartite',
    { nodeLabels: ['__ALL__'], relationshipTypes: ['__ALL__'] }
  )
  YIELD nodeId, score
  RETURN collect(nodeId)[0..10] AS topPlayers
```

```
}
CALL gds.nodeSimilarity.filtered.stream(
  'monopartite',
  {
    sourceNodeFilter: topPlayers,
    targetNodeFilter: '__ALL__',
    similarityMetric: 'JACCARD',
    topK: 20,
    similarityCutoff: 0.1
  }
)
YIELD node1, node2, similarity
RETURN
  gds.util.asNode(node1).fullGivenName AS player,
  gds.util.asNode(node2).fullGivenName AS similarPlayer,
  similarity
ORDER BY similarity DESC, player;
```

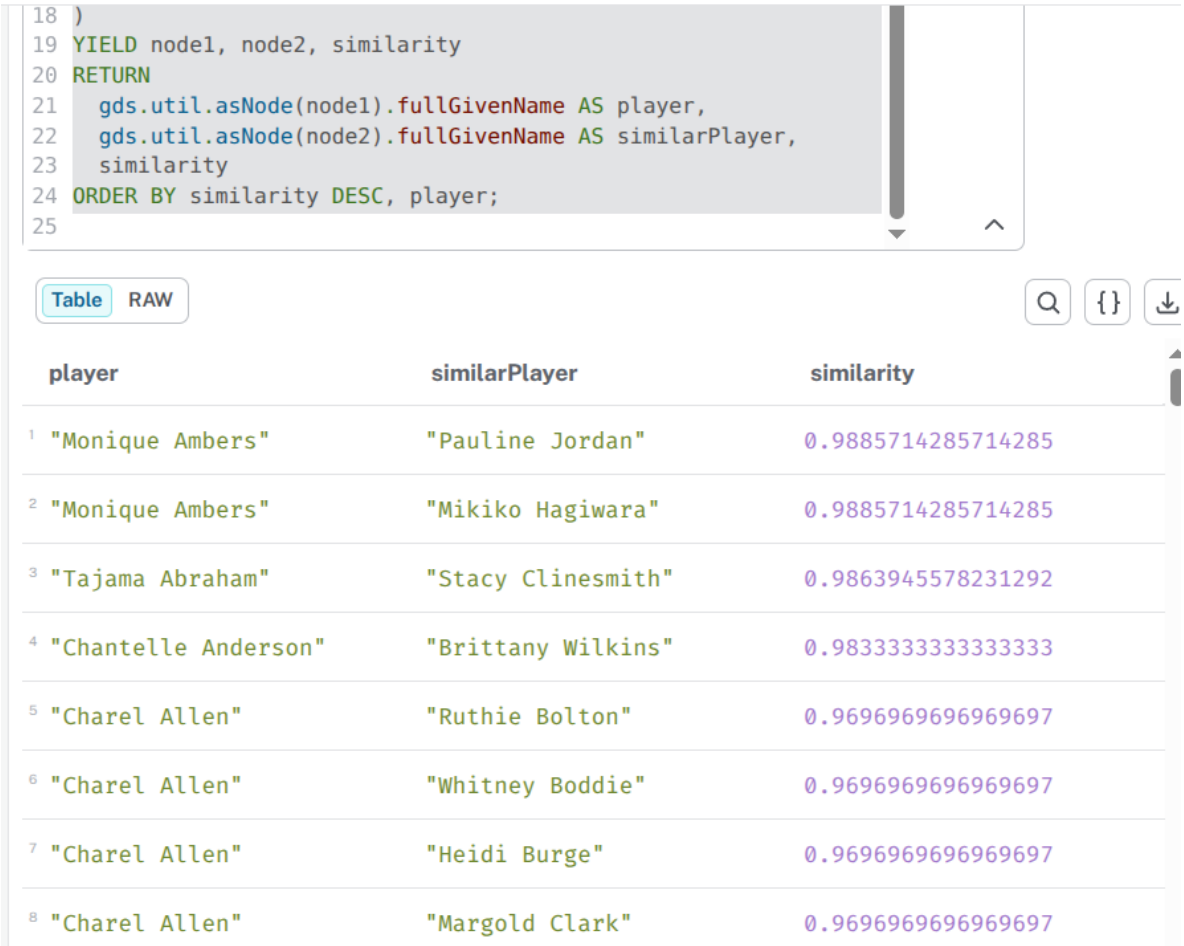

Figure 9: We compute the Jaccard on the monopartite projection.

The results we get (Fig 9) provide us with a good visualisation of players sharing a common history. For example, at the very top, we have Monique Ambers and Pauline Jordan, who both did their careers in the Phoenix Mercury and the Sacramento Monarchs.

5.1.2 Overlap Similarity

To compute the overlap similarities, we just need to change the last part of both queries.

```
// ...
CALL gds.nodeSimilarity.filtered.stream(
  'bipartite',
  {
    sourceNodeFilter: topPlayers,
    targetNodeFilter: 'Player',
    similarityMetric: 'OVERLAP',
    topK: 20,
    similarityCutoff: 0.1
  }
)
YIELD node1, node2, similarity
RETURN
  gds.util.asNode(node1).fullGivenName AS player,
  gds.util.asNode(node2).fullGivenName AS similarPlayer,
  similarity
ORDER BY similarity DESC, player;
```

As we can see in Fig. 10, all returned nodes have a similarity score of 1.0. Why this difference with the Jaccard?

Both algorithms compute a different thing. Say one player node played in 2 teams, in which another player node who played in 10 teams also played. The Jaccard similarity between the two would be $\frac{2}{10} = 0.2$, while the overlap would be $\frac{2}{2} = 1$, because the first player's neighbors are fully contained in the second player.

```
  'bipartite',
  {
    sourceNodeFilter: topPlayers,
    targetNodeFilter: 'Player',
    similarityMetric: 'OVERLAP',
    topK: 20,
    similarityCutoff: 0.1
  }
)
```

Table RAW

	player	similarPlayer	similarity
1	"Danielle Adams"	"Morenike Atunrase"	1.0
2	"Danielle Adams"	"LaQuanda Barksdale"	1.0
3	"Danielle Adams"	"Valeriya Berezhynska"	1.0

Figure 10: We compute the Jaccard on the monopartite projection.

5.2 Community Analysis

A community can be loosely defined as a group of nodes that are more strongly connected to each other than to the rest of the graph.

This allows us to detect communities, meaning group of nodes forming groups within the graph.

One such algorithm is Louvain. Louvain is an algorithm meant for monopartite graphs only. As such, we will only run it on our `monopartite` projection.

5.2.1 Louvain

```
CALL gds.louvain.write(
  'monopartite',
  {
    writeProperty: 'communityId'
  }
)
YIELD communityCount, modularity, ranLevels, nodePropertiesWritten;
```

Notice this part `WHERE 'Player' IN labels(n)`, whic filters out non-Player nodes.

Figure 11: Detecting communities

As we can see in figure 11, Louvain identified 92 communities. They reveal what teams the players played in, as this is based on the graph's relationships (which in our mono-partite projection only reveals the teams the players played in).

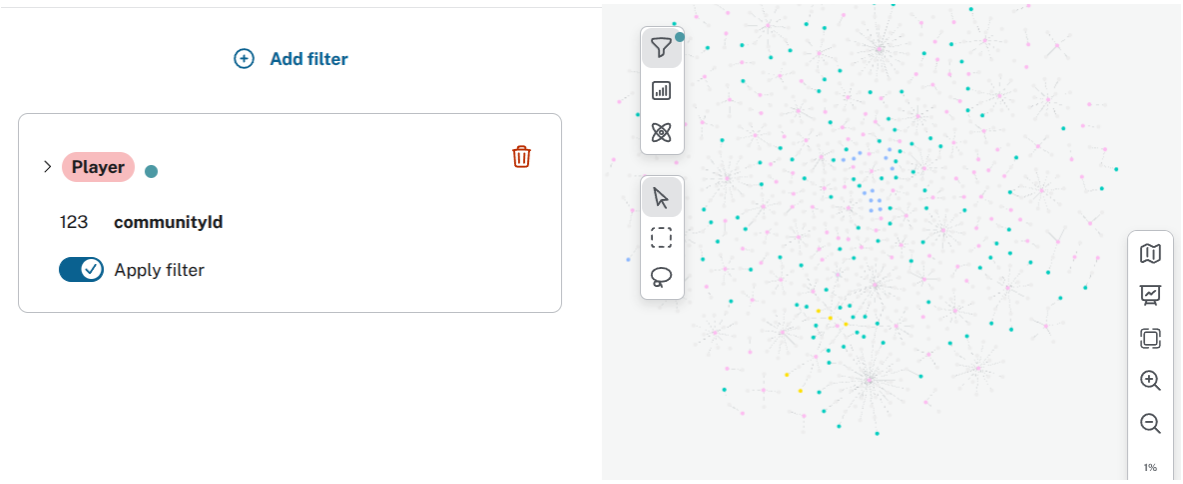

Figure 12: Detecting communities

As we can see in Fig. 12, we can then filter the view by community ID. (Community IDs go above 92 because they are not sequential.)

Future work may include creating new relationships between Players (for instance, if they went to the same high school or college), and run Louvain specifically on these relationships or on all relationships together, to see the impact it has on communities. However, Louvain is not deterministic. Change will happen every time it is run.

5.2.2 Connected Components

Connected Components is an algorithm useful for detecting independent communities in a graph. It is also an algorithm we can run on our bipartite graph.

```
CALL gds.wcc.stream('bipartiteProj')
YIELD nodeId, componentId
RETURN gds.util.asNode(nodeId).fullGivenName AS Player,
       componentId
ORDER BY componentId
```

The vast majority of the returned Players belong to "componentId" number 0. Others however belong to a separate "componentId" in which they are the only Player.

What this means is that most players belong to the same component (it is possible to access any player from any other player by following a stream of relationships). Some others however are completely isolated, because they are not linked to any other player. (Figure 13.)

	Player	componentId
839	"Sharnee' Zoll"	0
840	null	0
841	null	0
842	null	0
843	"Michael Adams"	5
844	"Richie Adubato"	6
845	"Brian Agler"	7
846	"Sonny Allen"	15
847	"Angela Beck"	52

Figure 13: Most nodes are connected together, apart from a few isolated ones.

5.3 Degree Centrality

First, let's answer the question: What is Degree Centrality? Well, according to Neo4j's documentation[8]:

The Degree Centrality algorithm can be used to find popular nodes within a graph. The degree centrality measures the number of incoming or outgoing (or both) relationships from

[8] https://neo4j.com/docs/graph-data-science/current/algorithms/degree-centrality/

a node, which can be defined by the orientation of a relationship projection. If a projection contains directed relationships, only outgoing relationships of a node are counted (the out-degree).

Because we converted all relationships to undirected ones, we don't have to worry about ingoing relationships not being counted.

Running the Degree Centrality algorithm on the bipartite projection will give us the most central nodes, *i.e.* the nodes with the highest number of relationships.

```
CALL gds.degree.stream(
  'bipartite',
  {
    relationshipTypes: ['PLAYED_IN']
  }
) YIELD nodeId, score
WHERE 'Player' IN labels(gds.util.asNode(nodeId))
RETURN gds.util.asNode(nodeId).fullGivenName, score
ORDER BY score DESC
```

With this query, we unsurprisingly get teams as the most central. (A player cannot match the number of relationships a team has.)

```
CALL gds.degree.stream(
  'bipartite',
  {
    relationshipTypes: ['PLAYED_IN']
  }
) YIELD nodeId, score
RETURN gds.util.asNode(nodeId).tId, score
ORDER BY score DESC
```

Table RAW

	gds.util.asNode(nodeId).tId	score
1	"PHO"	117.0
2	"WAS"	102.0
3	"LAS"	100.0

Figure 14: Most central teams.

At the very top are PHO, WAS, and LAS. (Fig. 14)

We will now compute the most central players only. (Fig. 15)

```
CALL gds.degree.stream(
  'bipartite',
  {
    relationshipTypes: ['PLAYED_IN']
  }
) YIELD nodeId, score
WHERE 'Player' IN labels(gds.util.asNode(nodeId))
RETURN gds.util.asNode(nodeId).fullGivenName, score
ORDER BY score DESC
```

```
CALL gds.degree.stream(
  'bipartite',
  {
    relationshipTypes: ['PLAYED_IN']
  }
) YIELD nodeId, score
WHERE 'Player' IN labels(gds.util.asNode(nodeId))
RETURN gds.util.asNode(nodeId).fullGivenName, score
ORDER BY score DESC
```

Table RAW

	gds.util.asNode(nodeId).fullGivenName	score
1	"Taj McWilliams-Franklin"	9.0
2	"Kristen Rasmussen"	8.0
3	"Stacey Lovelace"	8.0
4	"Sheri Sam"	8.0
5	"Kelly Miller"	7.0

Figure 15: Most central players.

In our graph, they would correspond to players with the richest team history.
But how does it differ from the same query run on our monopartite graph?

```
CALL gds.degree.stream(
  'bipartite',
  {
    relationshipTypes: ['PLAYED_IN']
  }
) YIELD nodeId, score
WHERE 'Player' IN labels(gds.util.asNode(nodeId))
RETURN gds.util.asNode(nodeId).fullGivenName, score
ORDER BY score DESC
```

As we can see in Fig. 16, we get roughly the same results as for the bipartite projection but with some differences. This is because the relationships in both projections do not mean the same thing, and so is the degree centrality. Degree Centrality in the bipartite graph simply and plainly indicates the number of teams the player played in, while in the monopartite graph, it is an indicator of both the number of teams and the number of players within those theams.

5.4 Link Prediction

We will only consider the top 10 players with the highest degree.

Let's see the number of teams they both share in their history **for our bipartite graph**:

```
// 1) Get top 10 Player nodes by degree (in the bipartite projection)
CALL {
```

```
1 CALL gds.degree.stream('monopartite') YIELD nodeId, score
2 RETURN gds.util.asNode(nodeId).fullGivenName, score
3 ORDER BY score DESC
```

Table RAW

	gds.util.asNode(nodeId).fullGivenName	score
1	"Tamara Moore"	1146.0
2	"Taj McWilliams-Franklin"	1134.0
3	"Kelly Miller"	1132.0
4	"Kristen Rasmussen"	1076.0
5	"Kiesha Brown"	1066.0
6	"Stacey Lovelace"	986.0
7	"Sheri Sam"	948.0
8	"Olympia Scott"	902.0

Figure 16: Most central players in the monopartite graph.

```
  CALL gds.degree.stream('bipartite', { relationshipTypes: ['PLAYED_IN'] })
  YIELD nodeId, score
  WITH nodeId, score
  WHERE gds.util.asNode(nodeId):Player
  ORDER BY score DESC
  LIMIT 10
  RETURN collect(gds.util.asNode(nodeId)) AS topPlayers
}

// 2) Generate all unordered pairs among the top 10
UNWIND range(0, size(topPlayers) - 2) AS i
UNWIND range(i + 1, size(topPlayers) - 1) AS j
WITH topPlayers[i] AS p1, topPlayers[j] AS p2

// 3) Call the pairwise link-prediction functions
RETURN
  p1.fullGivenName AS Player1,
  p2.fullGivenName AS Player2,
  gds.linkprediction.commonNeighbors(p1, p2, {
    relationshipQuery: "PLAYED_IN",
    direction: "BOTH"
  }) AS CommonNeighbors,
  gds.linkprediction.adamicAdar(p1, p2, {
      relationshipQuery: "PLAYED_IN",
      direction: "BOTH"
  }) AS AdamicAdar
ORDER BY CommonNeighbors DESC, AdamicAdar DESC
```

This is a big query, so let's try to understand it.

The first part of the query simply returns the top 10 players based on their centrality (*i.e.* their degree, the number of relationships they have) in the bipartite graph.

The second part of the query generates all possible pairs of different nodes from the previous result.

Finally, the last part simply calls two procedures from GDS: `commonNeihgbors` and |adamicAdar|.

`commonNeighbors` is self-explanatory, and we've already seen it. Adamic-Adar deserves an explanation. It calculates a score of similarity between two nodes based on the commonality of their connections. The formula used is:

$$A(x,y) = \sum_{u \in N(x) \cap N(y)} \frac{1}{\log |N(u)|}$$

For both Adamic-Adar and Common Neighbors, the higher both scores are, the higher the likelihood of a relationship forming between the two nodes.

We get 2 as a result.

5.5 Path finding

For our graph, path finding does not indicate to us how similar nodes are between each other. However, they can help us understand the relations between them.

For instance, let's focus on two nodes with no relationship through a team: "Stacey Lovelace" and "Adrienne Goodson". It can thus be interesting to see how we go from one node to the other.

```
MATCH p = (p1:Player {fullGivenName:"Taj McWilliams-Franklin"})
          -[:PLAYED_IN*1..4]-
          (p2:Player {fullGivenName:"Kristen Rasmussen"})
RETURN p;
```

The result we get gives us a player between the two ("Markita Aldridge") who played in both teams. (Fig. 17)

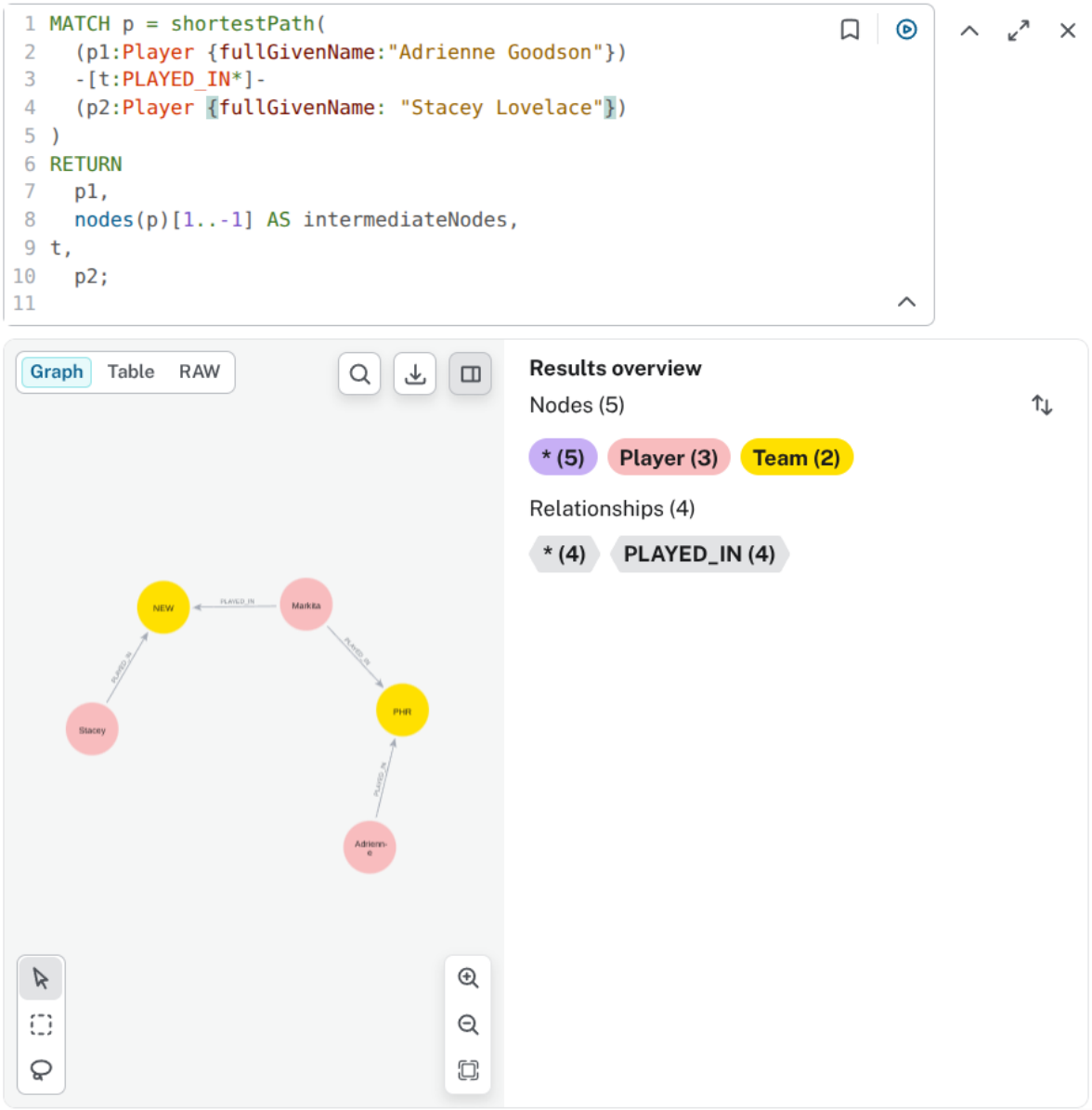

Figure 17: Shortest path between the two players.

6 Conclusion

In this project, we modeled and analysed a women's basketball dataset using Neo4j and its Graph Data Science (GDS) library. Starting from raw JSON data, we performed basic EDA, identified the main entities, and transformed the dataset into multiple CSV files suitable for graph import and analysis.

By creating in-memory projections, we explored both bipartite (Players–Teams) and monopartite (Players only) graphs, highlighting how reducing partiteness enables advanced algorithms at the cost of introducing artificial relationships. We applied similarity measures, community detection, connected components, and degree centrality to uncover patterns in player careers and team histories.

Overall, this work shows the relevance of graph-based analysis for understanding complex relational data. Future extensions could include adding new types of relationships or temporal information to further refine similarity, community structure, and link prediction results.